Assignment 3: Thread Scaling in Shared-Memory Matrix Multiplication

Abel Castro Villar
Data Science and Engineering — Big Data (Academic Year 2026–2027)
Universidad de Las Palmas de Gran Canaria (ULPGC)
GitHub Repository: https://github.com/abelcasv/Assignment03_Abel

November 2026

Abstract

This study investigates how OpenMP thread count (p) affects the execution time and memory footprint of dense square matrix multiplication ($n \times n$, where n is the matrix side length), and identifies when shared-memory parallel execution provides a performance advantage on a hybrid x86-64 processor. A C11 loop-interchanged (i-k-j) kernel is compared against its exact sequential counterpart on identical IEEE 754 `binary64` inputs across six dimensions ($n \in \{250, 500, 750, 1000, 1500, 2000\}$) and eight requested OpenMP team sizes ($p \in \{1, 2, 4, 8, 10, 12, 16, 24\}$) on an Intel Core i5-1335U (10 physical cores: 2 P-cores + 8 E-cores; 12 logical processors). Each configuration executes one untimed warm-up followed by five measured repetitions, summarised via medians and interquartile ranges (IQR). At $n = 2000$, median kernel runtime decreases from 15.2589 s sequentially to 1.3768 s with $p = 12$ threads, achieving a matched speedup of $S(12) = 11.08\times$ and a parallel efficiency of $E(12) = 92.4\%$, whereas the single-thread OpenMP configuration ($p = 1$) is 7.3% slower (16.3745 s, $S(1) = 0.93\times$) than the sequential baseline. Scaling is non-monotonic across intermediate sizes and oversubscribed regimes ($p \in \{16, 24\}$), and a secondary cache-blocked algorithm ($B_s = 32$) exhibits a distinct thread-scaling profile (4.7202 s sequentially down to 0.6026 s at $p = 8$ and 0.5826 s at $p = 24$). The empirical evidence supports parallel execution for large matrices on this host while highlighting the influence of runtime variability and hybrid-core scheduling.

1 Introduction

Dense matrix multiplication ($C = A \times B$) is a foundational workload for evaluating shared-memory parallelism because its arithmetic complexity scales cubically as $2n^3$ floating-point operations (FLOPs), whereas storage for three dense $n \times n$ IEEE 754 `binary64` matrices grows quadratically as $3 \times 8n^2 = 24n^2$ bytes (where n denotes the side length of an $n \times n$ square matrix). In accordance with the Assignment 3 protocol [1], this study compares an optimised sequential baseline against an equivalent OpenMP parallel implementation, evaluates thread scaling from single-thread execution up to hardware oversubscription, validates numerical correctness prior to timing, and reports both execution time and memory metrics with full reproducibility artifacts.

1.1 Research Question and Hypotheses

- **Research Question:** *How does OpenMP thread count p affect execution time, run-to-run variability, parallel efficiency, and memory usage for dense square matrix multiplication across $n \in \{250, 500, 750, 1000, 1500, 2000\}$, and under which workload sizes does parallel execution outperform its matched sequential baseline on a hybrid 10-core / 12-thread processor?*
- **Hypotheses (Pre-Measurement Predictions):** Prior to experimentation, we hypothesised that: (1) at $p = 1$, the OpenMP implementation would incur measurable runtime overhead relative to the pure sequential baseline due to parallel-region fork/join management; (2) multi-thread execution would yield substantial speedups for large dimensions ($n \geq 1000$) where arithmetic work amortises thread synchronisation; and (3) parallel efficiency $E(p)$ would decline as p transitions from the 2 Performance-cores to the 8 Efficient-cores and degrade further under oversubscription ($p > 12$ logical processors). These predictions are tested empirically below without assuming monotonic scaling a priori.

2 Methodology

2.1 Implementations and Shared Input Generation

To ensure a fair, controlled comparison, the primary sequential baseline (`seq_ikj`) and its parallel counterpart (`omp_ikj`) share the exact same C11 row-major 1D storage layout (`double*`), numeric type (`double`), compiler flags, and i-k-j loop nesting:

- **Primary Sequential and OpenMP Kernels (`seq_ikj` vs. `omp_ikj`):** Both kernels hoist $r = A_{i,k}$ into a scalar register and stream row k of B and row i of C with unit stride (stride-1). In `omp_ikj`, the outermost row loop $i \in [0, n-1]$ is parallelised via `#pragma omp parallel` and `#pragma omp for schedule(static)`. Because each outer iteration i writes exclusively to row i of C while A and B are read-only, shared-memory access is completely race-free without requiring mutex locks, atomic operations, or critical sections.
- **Secondary Blocked Algorithm Comparison (`seq_tiled_b32` vs. `omp_tiled_b32`):** A second pair of kernels partitions the iteration space into $B_s \times B_s$ sub-blocks ($B_s = 32$) to improve cache reuse [2]. In `omp_tiled_b32`, OpenMP statically distributes the outer tile-row loop `ii` $\in \{0, 32, \ldots, n-1\}$ ($\lceil n/32 \rceil$ tile rows). In strict compliance with the rubric, `omp_tiled_b32` is evaluated exclusively against its own matched sequential counterpart (`seq_tiled_b32`) as a secondary algorithmic comparison and never mixed with `seq_ikj`. A canonical i-j-k implementation (`canonical_ijk`) is used solely during correctness verification.
- **Input Generation and Checksum Verification:** Shared input pairs (A_n, B_n) for $n \in \{250, 500, 750, 1000, 1500, 2000\}$ and 4×4 validation fixtures were generated via `src/generate_data.py` using Python's deterministic Mersenne Twister PRNG (`random.Random(42)`) [3]. Each entry is drawn from $[0.0, 1.0)$ and formatted to six decimal places in CSV (yielding stored values in $[0.000000, 1.000000]$ parsed as IEEE 754 `binary64`). The generator outputs `data/SHA256SUMS.txt` to verify input integrity across runs.

2.2 Host Environment and Thread Configuration

Table 1 summarises the hardware and software environment captured automatically via `src/capture_environmen` and runtime telemetry (`results/environment_*.txt`).

Table 1: Recorded hardware, compiler, OpenMP runtime, and analysis environment.

Subsystem	Recorded Specification and Runtime Settings
Processor (CPU)	13th Gen Intel Core i5-1335U: 10 physical cores (2 P-cores + 8 E-cores), 12 logical processors [4]
Cache Hierarchy	L1d: 48 kB/P-core (32 kB/E-core); L2: 1.25 MB/P-core (2 MB/E-cluster); L3: 12 MB shared [4]
Memory & OS	15.63 GiB physical RAM (16 GB nominal); Windows 11 64-bit (Version 10.0.26300, Build 26300)
Compiler & Flags	MSYS2 UCRT64 GCC 16.1.0; `-Wall -Wextra -O3 -std=c11 -fopenmp -march=native -lpsapi`
OpenMP Runtime	OpenMP 5.2 (`_OPENMP=202111`) [5]; `omp_set_dynamic(0)`; `OMP_PROC_BIND=false`; `OMP_PLACES=unset`
Tested Team Sizes	$p \in \{1, 2, 4, 8, 10, 12, 16, 24\}$; actual team size verified via `omp_get_num_threads()` ($p > 12$ oversubscribed)
Analysis Stack	Python 3.12.14; NumPy 2.5.4; pandas 3.0.1; Matplotlib 3.11.2

Dynamic thread adjustment was explicitly disabled (`omp_set_dynamic(0)`) and actual team sizes were recorded inside every parallel region via `omp_get_num_threads()`, confirming that requested and actual thread counts matched identically in all observations. Because `OMP_PROC_BIND=false` and `OMP_PLACES` was unset, the Windows scheduler managed thread placement across P-cores and E-cores dynamically. Team sizes $p \in \{16, 24\}$ exceed the 12 hardware execution contexts and are explicitly flagged as oversubscribed (`oversubscribed=1`).

2.3 Timing Boundary, Metrics, Correctness Validation, and Resource Budgets

- **Timing Boundary and Output Consumption:** CSV parsing (`load_matrix`) and heap allocation (`malloc` of A, B, C) occur strictly outside the timed region. A monotonic high-resolution timer (Windows `QueryPerformanceCounter`) brackets a single kernel call, which

includes output zero-initialisation (`memset` of C) and, for OpenMP kernels, parallel team creation/synchronisation and static loop scheduling overhead. Immediately after the timer stops, all n^2 elements of C are summed into a `volatile double result_sink` outside the timed window to guarantee full execution and prevent dead-code elimination.

- **Summary Statistics and Scaling Metrics:** Every configuration executes 1 untimed warm-up followed by 5 timed repetitions. Let $T_{\text{seq}}(n)$ denote the **median** execution time of the sequential baseline for dimension n, and let $T_p(n)$ denote the **median** execution time of the matched parallel implementation with p threads. Speedup $S(p)$, parallel efficiency $E(p)$, and single-thread OpenMP relative difference ($\Delta_{p=1}$) are defined as:

$$S(p) = \frac{T_{\text{seq}}(n)}{T_p(n)}, \qquad E(p) = \frac{S(p)}{p} = \frac{T_{\text{seq}}(n)}{p \cdot T_p(n)}, \qquad \Delta_{p=1} = \frac{T_{\text{omp}}(n,1) - T_{\text{seq}}(n)}{T_{\text{seq}}(n)} \times 100\% \quad (1)$$

  Non-parametric dispersion is reported via the interquartile range (IQR $= Q_3 - Q_1$, using 25th and 75th percentiles).
- **Correctness Validation and Tolerances:** Before timing, all sequential and OpenMP kernels are verified against 4×4 analytical ($A \times B = C_{\text{expected}}$), identity ($A \times I = A$), and zero ($A \times 0 = 0$) fixtures, followed by cross-validation across all $p \in \{1, 2, 4, 8, 10, 12, 16, 24\}$ at $n = 250$ against `seq_ikj`. Comparisons enforce $|x - y| \leq \epsilon_{\text{abs}} + \epsilon_{\text{rel}}|y|$ with $\epsilon_{\text{abs}} = \epsilon_{\text{rel}} = 10^{-4}$. These tolerances are appropriate because CSV inputs are rounded to 6 decimal places ($\pm 5 \times 10^{-7}$) and compiler auto-vectorisation (`-O3 -march=native`) and 32×32 cache tiling alter the IEEE 754 floating-point summation order across up to $n = 2000$ terms. All validation checks passed.
- **Resource Budgets and Memory Telemetry Limitations:** Prior to execution, we set a preventive three-array structural memory budget of 2048.0 MiB ($24n^2$ bytes) and a per-repetition reactive stop threshold of 120.0 s (if a completed call exceeded 120 s, that sample would be retained and remaining repetitions for that configuration skipped). The largest tested dimension was $n = 2000$ (91.55 MiB structural memory; maximum median runtime 16.3745 s), which satisfied both budgets with zero timeouts; larger dimensions were not attempted. Process memory is sampled via Windows `GetProcessMemoryInfo` (`PeakWorkingSetSize`) [6]. Because all configurations execute within a single process and `PeakWorkingSetSize` is a cumulative process-lifetime high-water mark, CSV rows do not represent independent per-configuration peaks; therefore, both the analytical three-array structural footprint (`structural_memory_mib`) and the overall process peak working set are reported.

3 Results

The validated dataset (`results/parallel_results.csv`) comprises 540 raw measurements across 108 configurations (6 sizes × 2 algorithms × [1 sequential + 8 OpenMP thread counts] × 5 repetitions), with zero excluded or missing observations.

3.1 Primary Thread-Scaling Comparison and Single-Thread OpenMP Overhead

Table 2 reports the matched comparison between `seq_ikj`, single-thread `omp_ikj` ($p = 1$), and 12-thread `omp_ikj` ($p = 12$, matching the 12 logical processors) across all six matrix dimensions, alongside the analytical structural memory footprint. Figure 1 illustrates the scaling trajectories across n.

At the largest tested size ($n = 2000$), 12 OpenMP threads reduce median execution time from 15.2589 s to 1.3768 s, yielding a matched speedup of $S(12) = 11.08\times$ (92.4% parallel efficiency). At smaller and intermediate sizes, speedups at $p = 12$ range from $2.55\times$ ($n = 250$) and $3.77\times$ ($n = 500$) to $5.93\times$ ($n = 1000$) and $3.89\times$ ($n = 1500$). Scaling across n is not strictly monotonic because the sequential baseline at $n = 1000$ exhibits a wide interquartile range (IQR = 0.6008 s), which inflates the apparent $n = 1000$ speedup relative to $n = 1500$.

Comparing `omp_ikj` at $p = 1$ against `seq_ikj` reveals that OpenMP single-thread execution is slower in five of the six dimensions: +0.9% at $n = 250$, +7.5% at $n = 500$, +12.8% at $n = 750$,

Table 2: Primary *i-k-j* scaling across matrix dimensions n: sequential baseline, OpenMP $p = 1$ overhead, and OpenMP $p = 12$ performance (medians and [IQR] over 5 repetitions; overall process peak working set is 97.36 MiB).

Size (n)	Struct. Mem (MiB)	Sequential `ikj` Median [IQR] (s)	OpenMP ($p = 1$) Median [IQR] (s)	$p = 1$ Diff. ($\Delta_{p=1}$)	OpenMP ($p = 12$) Median [IQR] (s)	Speedup $S(12)$	Efficiency $E(12)$
250	1.43	0.0053 [0.0002]	0.0053 [0.0002]	+0.9%	0.0021 [0.0003]	2.55×	21.2%
500	5.72	0.0595 [0.0051]	0.0640 [0.0029]	+7.5%	0.0158 [0.0025]	3.77×	31.5%
750	12.87	0.2512 [0.0044]	0.2834 [0.0252]	+12.8%	0.0631 [0.0076]	3.98×	33.2%
1000	22.89	1.2341 [0.6008]	0.9896 [0.0574]	−19.8%	0.2079 [0.0046]	5.93×	49.5%
1500	51.50	3.2699 [0.1017]	6.5567 [0.2289]	+100.5%	0.8397 [0.0973]	3.89×	32.5%
2000	91.55	15.2589 [1.7478]	16.3745 [4.6956]	+7.3%	**1.3768** [0.0792]	**11.08**×	**92.4%**

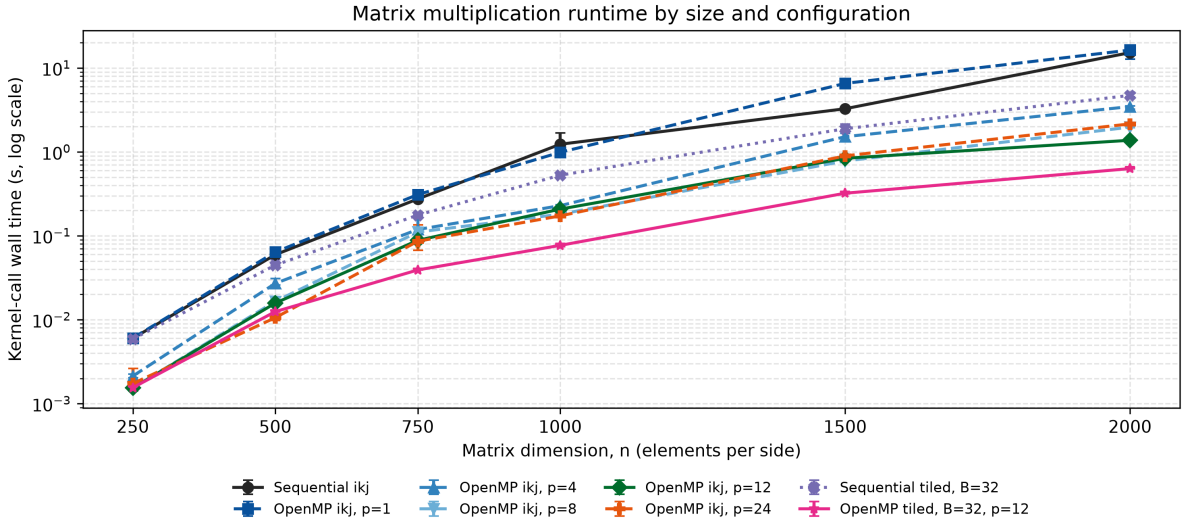


Figure 1: Runtime medians and interquartile ranges (IQR) by matrix dimension n for selected sequential and OpenMP configurations (logarithmic vertical axis).

+100.5% at $n = 1500$, and +7.3% slower at $n = 2000$ (16.3745 s vs. 15.2589 s, $S(1) = 0.93\times$). Only at $n = 1000$ is the $p = 1$ median lower (−19.8%), driven by the elevated sequential baseline IQR in that sample. Thus, single-thread differences reflect both OpenMP runtime overhead and system-state variability rather than a constant additive cost.

3.2 Strong Scaling Across Thread Counts and Secondary Blocked-Kernel Comparison

Table 3 presents both the primary `omp_ikj` scaling and the secondary cache-blocked (`omp_tiled_b32`) comparison across all eight team sizes at $n = 2000$.

Table 3: Strong scaling at $n = 2000$ across requested OpenMP team sizes $p \in \{1, 2, 4, 8, 10, 12, 16, 24\}$ for the primary kernel (`omp_ikj` vs. `seq_ikj` = 15.2589 s [1.7478 s]) and secondary blocked kernel (`omp_tiled_b32` vs. `seq_tiled_b32` = 4.7202 s [0.2528 s]).

Threads (p)	Oversub. ($p > 12$)	Primary `omp_ikj` Median [IQR] (s)	Primary $S(p)$	Primary $E(p)$	Blocked `omp_tiled_b32` Median [IQR] (s)	Blocked $S_{\text{tiled}}(p)$	Blocked $E_{\text{tiled}}(p)$
1	No	16.3745 [4.6956]	0.93×	93.2%	4.7891 [0.2330]	0.99×	98.6%
2	No	5.8807 [3.7609]	2.59×	129.7%	2.5015 [0.1118]	1.89×	94.3%
4	No	3.4712 [0.2777]	4.40×	109.9%	1.2981 [0.0560]	3.64×	90.9%
8	No	1.9744 [0.0229]	7.73×	96.6%	0.6026 [0.0170]	7.83×	97.9%
10	No	1.5443 [0.1340]	9.88×	98.8%	0.7208 [0.0558]	6.55×	65.5%
12	No	**1.3768** [0.0792]	**11.08**×	92.4%	0.6929 [0.0445]	6.81×	56.8%
16	Yes	2.3040 [0.9285]	6.62×	41.4%	0.6987 [0.0706]	6.76×	42.2%
24	Yes	2.1583 [0.1481]	7.07×	29.5%	**0.5826** [0.0235]	**8.10**×	33.8%

In the primary `omp_ikj` kernel at $n = 2000$ (Table 3), speedup rises from 0.93× ($p = 1$) to 9.88× at $p = 10$ (98.8% efficiency) and peaks at 11.08× at $p = 12$ (92.4% efficiency). When oversubscribed beyond the 12 hardware threads, performance drops to 6.62× at $p = 16$ (2.3040 s, $E = 41.4\%$) and 7.07× at $p = 24$ (2.1583 s, $E = 29.5\%$). Figure 2 visualises these scaling trajectories.

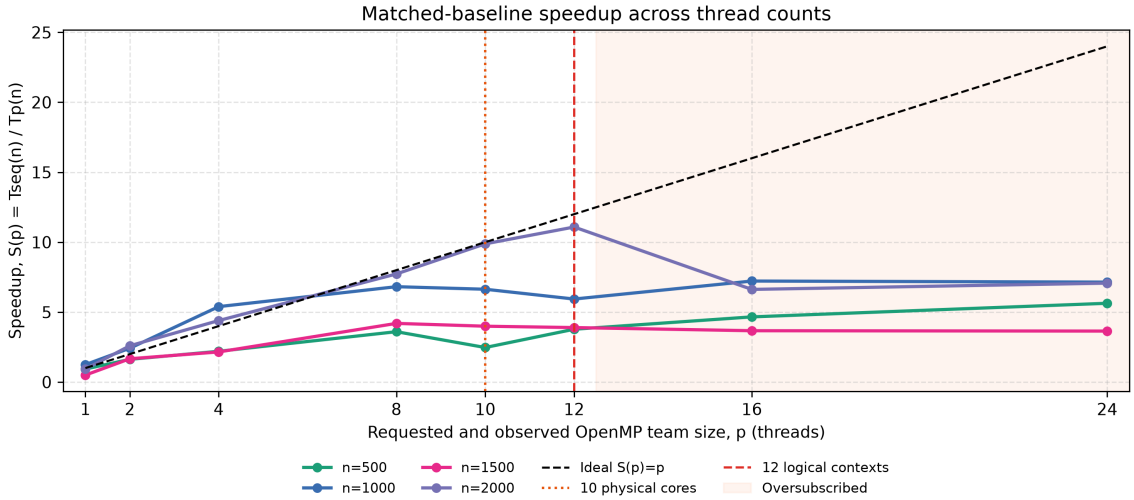


Figure 2: Matched-baseline speedup $S(p) = T_{\text{seq}}(n)/T_p(n)$ for the primary `omp_ikj` kernel across thread counts p. The dashed line denotes ideal linear speedup; vertical lines mark physical cores (10) and logical processors (12).

Notably, at $p = 2$ and $p = 4$, the observed medians yield apparent superlinear efficiencies (129.7% and 109.9%), as visualised in Figure 3.

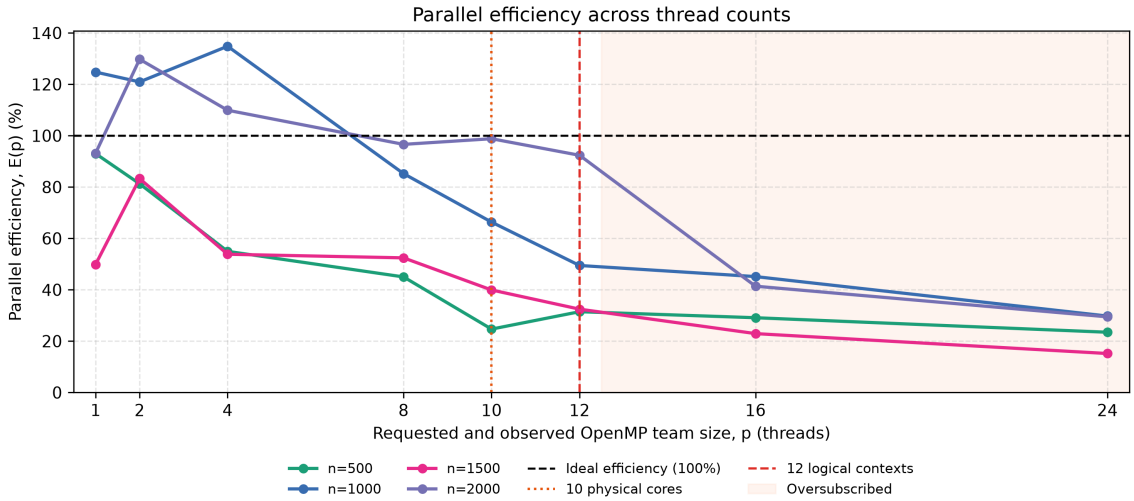


Figure 3: Parallel efficiency $E(p) = S(p)/p$ for the primary `omp_ikj` kernel. Values above 100% remain unclipped; shaded region denotes hardware oversubscription ($p > 12$ logical processors).

While team sizes, inputs, and timing boundaries were strictly verified, the $p = 2$ sample exhibits a wide IQR (3.7609 s) and the sequential baseline (15.2589 s, IQR = 1.7478 s) ran earlier in the thermal/frequency cycle without core pinning. Thus, these superlinear ratios are treated as empirical observations influenced by baseline variance, turbo-frequency boosting, or P-core scheduling rather than proof of a single hardware mechanism.

In the secondary blocked comparison at $n = 2000$ (Table 3, right columns), sequential cache blocking (`seq_tiled_b32`, 4.7202 s) is already 3.23× faster than sequential `seq_ikj` (15.2589 s) due to L1/L2 tile reuse. When parallelised against its own baseline, `omp_tiled_b32` reaches 0.6026 s at $p = 8$ ($S_{\text{tiled}}(8) = 7.83\times$, $E = 97.9\%$), slows slightly at $p = 10$ (0.7208 s) and $p = 12$ (0.6929 s), and

achieves its fastest observed median at $p = 24$ (0.5826 s, $S_{\text{tiled}}(24) = 8.10\times$). Because `omp_tiled_b32` statically distributes the outer tile-row loop `ii` ($\lceil 2000/32 \rceil = 63$ tile rows), $p = 8$ divides the 63 tile rows almost evenly (7–8 tile rows/thread), whereas $p = 10$ and $p = 12$ assign unequal chunk counts across heterogeneous P-cores and E-cores; at $p = 24$, smaller static chunks (2–3 tile rows/thread) allow faster cores to process multiple threads via OS context switching. Nevertheless, the 0.0200 s difference between $p = 8$ and $p = 24$ is small and does not support a general claim that oversubscription is beneficial.

Regarding memory, the three-array structural footprint scales from 1.43 MiB ($n = 250$) to 91.55 MiB ($n = 2000$, Table 2), well below the 2048 MiB budget. The cumulative process peak working set across the entire benchmark run reached 97.36 MiB, reflecting the 91.55 MiB arrays at $n = 2000$ plus ≈ 5.81 MiB of C runtime and OpenMP thread-pool stack overhead.

4 Discussion and Limitations

The controlled experiments directly answer the research question within the tested scope: shared-memory OpenMP parallelisation substantially accelerates matrix multiplication once matrix dimensions are large enough to amortise team overhead, achieving an 11.08× speedup at $n = 2000$ with $p = 12$ threads for `omp_ikj`, whereas $p = 1$ incurs a 7.3% slowdown at $n = 2000$ and up to +100.5% overhead at $n = 1500$.

Three methodological limitations bound these conclusions: (1) because `OMP_PROC_BIND=false` and hardware performance counters (e.g., L1/L2/L3 cache misses, core residency, and clock frequency telemetry) were not collected, the data establish empirical runtime trends but cannot isolate causal contributions of P-core vs. E-core scheduling or thermal throttling; (2) `PeakWorkingSetSize` is a process-lifetime high-water mark rather than an independent per-configuration peak; and (3) the study is bounded at $n = 2000$ with 5 repetitions on a single Windows host, and the optional SIMD vectorisation extension was not undertaken in the core study.

5 Conclusion

Using matched algorithms, identical `binary64` inputs, validated correctness checks, and five repetitions across 108 configurations on an Intel Core i5-1335U, this study establishes that: (1) OpenMP parallelisation of the *i-k-j* kernel reduces $n = 2000$ median runtime from 15.2589 s to 1.3768 s at $p = 12$ ($S(12) = 11.08\times$, $E(12) = 92.4\%$); (2) single-thread OpenMP ($p = 1$) is generally slower than pure sequential execution (+0.9% to +100.5% across five of six sizes); and (3) oversubscription ($p \in \{16, 24\}$) degrades primary-kernel speedup to 6.62 × –7.07×, while secondary 32 × 32 cache blocking reduces sequential $n = 2000$ runtime to 4.7202 s and reaches 0.6026 s–0.5826 s in parallel. All source code, SHA-256 input manifests, raw CSV measurements, environment logs, and plotting scripts are archived in the companion repository.

AI Assistance Declaration

In compliance with the course Generative AI policy, the author discloses that Gemini was used to draft the initial `CLAUDE.md` engineering protocol and prompt plan, Claude was used to assist with initial C11/Python implementation and documentation, and OpenAI Codex was used for an independent audit and subsequent refinements to the benchmark harness, data validation, reproducibility scripts, and report text. All code was compiled and executed locally by the author on genuine hardware, all measurements are authentic, and the author assumes full responsibility for defending the code, data, and conclusions.

References

[1] Big Data Course Team, "BIG DATA | Individual Assignments | Revised proposal 2026–2027," Assignment 3 and common experimental protocol, course document, ULPGC, 2026.

[2] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software (TOMS)*, vol. 34, no. 3, Art. 12, May 2008, doi: 10.1145/1356052.1356053.

[3] Python Software Foundation, "`random` — Generate pseudo-random numbers," Python 3.12 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/random.html. Accessed: Nov. 1, 2026.

[4] Intel Corporation, "Intel Core i5-1335U Processor Specifications," Intel ARK Product Specifications, 2026. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/232153/intel-core-i51335u-processor-12m-cache-up-to-4-60-ghz/specifications.html. Accessed: Nov. 1, 2026.

[5] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Version 5.2*, Nov. 2021. [Online]. Available: https://www.openmp.org/spec-html/5.2/openmp.html.

[6] Microsoft Corporation, "Process Memory Usage Information (`GetProcessMemoryInfo`)," Windows Win32 PSAPI Documentation, 2026. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/psapi/process-memory-usage-information. Accessed: Nov. 1, 2026.